SIMATS ENGINEERING
ASSESSMENT TOOL 1

Name:E.V.Mahendra Reddy(192425214)
COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

CODING CHALLENGE

Duration: 2hrs

1.Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. (5 marks)

Answer:

Reinforcement Learning (RL) is a framework in which an agent learns to make decisions by interacting with an environment through a continuous cycle of observation, action, and feedback, with the objective of maximizing cumulative reward over time.

- **Agent:** the learner/decision-maker that observes the environment and chooses actions (e.g., a robot, a game-playing program).
- **Environment:** everything external to the agent that responds to its actions and returns a new situation and a reward signal.
- **State (S):** a representation of the current situation of the environment at a given time step, which the agent uses to decide its next action.
- **Action (A):** the set of choices available to the agent; selecting an action transitions the environment to a new state.
- **Reward (R):** a scalar feedback signal indicating how good or bad the action taken in a given state was, guiding the agent toward desirable behaviour.
- **Interaction loop:** at each time step t, the agent observes state S_t, takes action A_t, and receives reward R_t+1 along with the next state S_t+1; this repeats until a terminal state or indefinitely.

Cumulative reward (the sum, often discounted, of rewards over a trajectory) is central because RL agents do not optimize immediate reward alone — they learn a policy that maximizes long-term return, which forces the agent to trade off short-term gains against long-term benefit and shapes exploration, planning, and credit-assignment decisions throughout learning.

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

Answer:

Many sequential decision-making problems can be formally modeled as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ), which assumes the Markov property — the next state and reward depend only on the current state and action, not on the full history.

- **State space (S):** the set of all possible situations the environment can be in.
- **Action space (A):** the set of all actions available to the agent in a given state; it can be discrete (finite choices) or continuous (real-valued controls).
- **Transition probability P(s′|s,a):** the probability of moving to next state s′ given that action a was taken in state s; it captures the (often stochastic) dynamics of the environment.

- **Reward function R(s,a,s′):** specifies the immediate numerical reward received after transitioning from s to s′ via action a, defining the goal of the task.
- **Discount factor (γ ∈ [0,1]):** weights future rewards relative to immediate ones; a value close to 0 makes the agent myopic, while a value close to 1 makes it far-sighted.

By modeling a problem as an MDP, RL algorithms can use dynamic programming, Monte Carlo, or temporal-difference methods to compute an optimal policy π* that maximizes the expected discounted cumulative reward.

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.
(5 marks)

Answer:

In RL, a policy π and value functions together define how an agent behaves and how it judges the quality of that behaviour.

- **Policy (π):** a mapping from states to actions (deterministic π(s)=a, or stochastic π(a|s)) that defines the agent's behaviour.
- **State-value function V^π(s):** the expected cumulative discounted reward obtainable starting from state s and following policy π thereafter — it evaluates how good a state is under a given policy.
- **Action-value function Q^π(s,a):** the expected cumulative discounted reward obtainable by taking action a in state s and thereafter following policy π — it evaluates how good a specific action is in a specific state.

- **Bellman Equation:** expresses value functions recursively in terms of immediate reward plus the discounted value of the successor state, e.g. $V^{\pi}(s) = \Sigma_a \pi(a|s) \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^{\pi}(s')]$, and similarly for Q^{π}.

The Bellman Equation is the mathematical backbone of RL: it allows value functions to be computed iteratively (value iteration, policy iteration, Q-learning) and enables the agent to evaluate the long-term benefit of actions rather than just their immediate reward, which is essential for finding an optimal policy $\pi^* = \text{argmax}_a Q^*(s,a)$.

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ε-greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

Answer:

The exploration–exploitation trade-off is the fundamental dilemma in RL: the agent must decide between exploiting the best action known so far (to maximize immediate reward) and exploring lesser-known actions (to discover potentially better long-term strategies). Excessive exploitation risks getting stuck in a suboptimal policy, while excessive exploration wastes reward on poor actions.

- **ε-greedy strategy:** with probability ε the agent picks a random action (exploration), and with probability $1-\varepsilon$ it picks the action with the highest estimated value (exploitation); ε is often decayed over time so the agent explores more early on and exploits more later — simple to implement but treats all non-greedy actions equally regardless of how promising they are.
- **Softmax (Boltzmann) strategy:** actions are chosen probabilistically in proportion to their estimated values

using a Boltzmann distribution controlled by a temperature parameter τ; higher-value actions are more likely to be chosen but lower-value actions still get a graded chance — this gives smoother, more informed exploration than ε-greedy since it distinguishes between a slightly worse and a much worse action.

In practice, softmax generally balances learning efficiency better in problems with many actions of varying quality, whereas ε-greedy is computationally cheaper and easier to tune, making the choice a trade-off between simplicity and exploration quality.

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

Answer:

Reward shaping is the technique of supplementing an environment's sparse or delayed reward signal with additional, carefully designed intermediate rewards that guide the agent toward desirable behaviour faster, without changing the ultimate optimal policy.

- **Accelerating learning:** by providing denser feedback (e.g., rewarding a robot for moving closer to a goal, not just for reaching it), the agent receives more frequent learning signals, which speeds up credit assignment and reduces the number of episodes needed to learn a good policy.
- **Potential-based shaping:** using a shaping function $F(s,a,s') = \gamma\Phi(s') - \Phi(s)$, based on a potential function Φ, guarantees that the optimal policy of the shaped MDP is the same as the original — this is the theoretically safe way to shape rewards.

- **Preventing suboptimal policies:** poorly designed shaping rewards can cause 'reward hacking', where the agent exploits the shaping signal instead of solving the real task (e.g., a boat-racing agent looping to collect shaping rewards instead of finishing the race); potential-based shaping and careful validation prevent this by preserving policy invariance.

Overall, well-designed reward shaping reduces sample complexity and helps the agent avoid long periods of undirected random exploration, while poorly designed shaping can bias the agent toward suboptimal or unintended behaviour.

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Answer:

Applying RL to real-world robotics introduces challenges that are largely absent in simulated environments, primarily because real robots operate under physical, safety, and time constraints.

- **Safety:** unlike simulation, exploratory (random) actions on a physical robot can cause damage to the robot, its surroundings, or humans; this requires constrained/safe exploration methods, action limits, or simulation-to-real (sim-to-real) transfer to avoid dangerous trial-and-error learning directly on hardware.
- **Sample efficiency:** real-world data collection is slow, costly, and wears down hardware, so RL algorithms that need millions of interactions (common in simulation) are impractical; this drives the need for sample-efficient

methods such as model-based RL, offline RL, or transfer learning from simulation.

- **Environment variability:** real environments are non-stationary and noisy — lighting, friction, sensor drift, and unpredictable obstacles change over time — making learned policies brittle unless the agent is trained with domain randomization or continually adapts online.

Together, these factors mean robotics RL research emphasizes safe exploration, simulation pretraining with sim-to-real transfer, and robust, sample-efficient algorithms rather than naive trial-and-error learning.

(5 marks)

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

Answer:

The discount factor γ ($0 \leq \gamma \leq 1$) controls how much an RL agent values future rewards relative to immediate ones, directly shaping the balance between short-term and long-term decision-making.

- **Low γ (close to 0):** the agent becomes myopic, heavily weighting immediate rewards and largely ignoring long-term consequences — useful when the task truly only cares about near-term outcomes, but risky for tasks requiring planning ahead.
- **High γ (close to 1):** the agent becomes far-sighted, valuing future rewards almost as much as immediate ones, which encourages strategies that sacrifice short-term reward

for greater long-term return, but can slow convergence and increase variance in return estimates.

- **Role in episodic tasks:** in tasks with a finite horizon (an episode that ends, e.g., a game or a single robotic task), γ still matters for weighting rewards within the episode; a γ near 1 is common since the episode naturally bounds the return, whereas in continuing (non-episodic) tasks $\gamma<1$ is essential to keep the cumulative return finite.

Choosing γ is therefore a key design decision: it must reflect how far into the future the task's success genuinely depends, and it directly affects the shape of the value function used in the Bellman equation.

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

Answer:

Model-free and model-based RL differ in whether the agent explicitly learns a model of the environment's dynamics before deciding how to act.

- **Model-free RL:** the agent learns a policy or value function directly from experience (e.g., Q-learning, SARSA, policy gradients) without ever learning the transition or reward functions; it is simpler to implement and works well in complex, high-dimensional environments, but typically requires far more interaction data (low sample efficiency) and adapts slowly to changes in the environment.
- **Model-based RL:** the agent first learns (or is given) a model of the environment's transition dynamics $P(s'|s,a)$ and

reward function, and then uses planning (e.g., value iteration, Monte Carlo Tree Search) to derive a policy; this is far more sample-efficient and can adapt quickly to new goals, but performance depends heavily on model accuracy, and learning a good model of complex or stochastic environments can itself be difficult and computationally expensive.

In dynamic environments, model-based RL is generally more effective when an accurate model can be learned or is available, since it can re-plan quickly when conditions change, whereas model-free RL is more robust to model-misspecification but needs extensive retraining to adapt, making it better suited when environment dynamics are too complex to model reliably.

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

Answer:

Q-learning is a model-free, off-policy algorithm that learns the optimal action-value function $Q^*(s,a)$ using the update rule: $Q(s,a) \leftarrow Q(s,a) + \alpha[\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \,]$, where α is the learning rate.

- **Example:** consider a simple grid-world where an agent at state s takes action 'right', receives reward $r = -1$ (a step cost) and lands in state s'. If $Q(s,\text{'right'}) = 2.0$, $\max_{a'} Q(s',a') = 3.0$, $\gamma = 0.9$, and $\alpha = 0.1$, then the updated value is $Q(s,\text{'right'}) = 2.0 + 0.1[-1 + 0.9(3.0) - 2.0] = 2.0 + 0.1(0.7) = 2.07$ — the estimate moves slightly closer to the observed return.

- **Convergence behaviour:** as the agent repeatedly visits state-action pairs and applies this update, the Q-values incrementally propagate reward information backward from goal states to earlier states; under conditions of sufficient exploration (every state-action pair visited infinitely often) and an appropriately decaying learning rate, Q-learning is guaranteed to converge to the optimal Q*(s,a), from which the optimal policy $\pi^*(s) = \text{argmax}_a Q^*(s,a)$ can be extracted.

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Answer:

On-policy and off-policy methods differ in whether the policy used to generate experience (the behaviour policy) is the same as the policy being learned and improved (the target policy).

- **On-policy learning:** the agent learns the value of the same policy it is currently using to act (e.g., SARSA), so exploration actions are incorporated into what is learned; this tends to produce safer, more stable learning that reflects actual behaviour, but is generally less sample-efficient because past experience from an old policy cannot be reused after the policy changes.
- **Off-policy learning:** the agent learns the value of an optimal (target) policy while following a different, often more exploratory, behaviour policy (e.g., Q-learning); this allows reuse of past experience (experience replay), learning from demonstrations, or learning multiple policies from the same data, giving much higher sample efficiency and flexibility.

In practice, off-policy methods are preferred when data is expensive or must be reused (e.g., robotics, deep RL with replay buffers), while on-policy methods are favoured when stability and safety of the exact behaviour being executed matters most, such as in safety-critical control tasks; the trade-off is largely one of sample efficiency versus stability/safety.

Code of Conduct:

I _____E.V.Mahendra Reddy(192425214) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and	Communicates well with minor	Communication lacks clarity or	Unable to communicate

		professionally with proper terminology	hesitation	confidence	effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately